

8.5 BONUS: ElevationYieldEngine.sol

Purpose: AI-driven staking engine for stablecoin yield using cross-chain leveraged DeFi strategies.

Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/access/Ownable.sol";

/**
 * @title ElevationYieldEngine
 * @notice AI-managed, cross-chain,
laddered-leverage staking strategy for
stablecoins.
 */
contract ElevationYieldEngine is Ownable {
    struct Strategy {
        string name;
        address targetProtocol;
        uint256 allocationBps; // Basis
points (1% = 100)
        uint256 leverageRatio; // e.g., 2 =
2x
        uint256 lockTime;    // time in
seconds
    }

    IERC20 public stableToken;
    address public rewardDistributor;
    Strategy[] public strategies;

    mapping(address => uint256) public
deposits;
    mapping(address => uint256) public
entryTimestamps;

    event Deposited(address indexed
user, uint256 amount);
    event Withdrawn(address indexed
user, uint256 amount);
    event StrategyAdded(string name);

    constructor(address _stableToken,
address _rewardDistributor) {
        stableToken =
IERC20(_stableToken);
        rewardDistributor =
_rewardDistributor;
    }

    function deposit(uint256 amount)
external {
        require(amount > 0, "Invalid
amount");

stableToken.transferFrom(msg.sender,
address(this), amount);
        deposits[msg.sender] += amount;
        entryTimestamps[msg.sender] =
block.timestamp;
        emit Deposited(msg.sender,
amount);
    }

    function withdraw(uint256 amount)
external {
        require(deposits[msg.sender] >=
amount, "Insufficient balance");
        deposits[msg.sender] -= amount;
        stableToken.transfer(msg.sender,
amount);
        emit Withdrawn(msg.sender,
amount);
    }

    function addStrategy(
        string calldata name,
        address targetProtocol,
        uint256 allocationBps,
        uint256 leverageRatio,
        uint256 lockTime
    ) external onlyOwner {
        strategies.push(Strategy({
            name: name,
            targetProtocol: targetProtocol,
            allocationBps: allocationBps,
            leverageRatio: leverageRatio,
            lockTime: lockTime
        }));
        emit StrategyAdded(name);
    }

    function rebalanceYieldStrategies()
external onlyOwner {
        // Placeholder for AI logic
        // Would typically fetch market
rates, rebalance pools, adjust leverage
    }

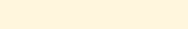
    function
getUserStakeDetails(address user)
external view returns (uint256 staked,
uint256 duration) {
        return (deposits[user],
block.timestamp -
entryTimestamps[user]);
    }

    function getStrategies() external view
returns (Strategy[] memory) {
        return strategies;
    }

    function
setRewardDistributor(address
_distributor) external onlyOwner {
        rewardDistributor = _distributor;
    }
}
```

Notes:

You can plug this directly into the treasury staking system—revenue is pooled here, diversified across strategies, and yield is paid to EOT or EUT holders.



Notein

Scan the QR code to download Notein

